Introducing Pandas

Pandas is a popular open source library for data analysis built on top of the Python programming language and it is the nucleus in a large Python ecosystem of data science tools. Pandas is heavily used for data analysis and cleaning and it is easily integrated with libraries for statistics, machine learning, web scraping, data visualization and more.

Import the Pandas library

Import the pandas library to get access to its features.

In [1]:
```
import pandas as pd
# pd.set_option('display.max_rows', None)
```

Importing a dataset

Pandas can import a variety of file types, each file type has its associated import method. We can use the read_csv method to tell Pandas to open up the *population.csv* file.

This method imports the CSV file's contents into an object called a **DataFrame**, a two-dimensional labeled data structure with columns of potentially different types.

In [2]:
```
url = 'https://raw.githubusercontent.com/GiorgioBar/pandas/main/datasets/populatio
population_df = pd.read_csv(url)
population_df.head()
```

Out[2]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
0	IT	Italy	1	males	Y0	0 years	205956
1	IT	Italy	2	females	Y0	0 years	195565
2	IT	Italy	9	total	Y0	0 years	401521
3	ITC	Nord-ovest	1	males	Y0	0 years	54122
4	ITC	Nord-ovest	2	females	Y0	0 years	51309

This DataFrame consists of seven columns and an index (the range of ascending numbers on the left side of the DataFrame). Index labels serve as identifiers for rows of data. We can set any column as the index of the DataFrame, if we do not explicitly tell pandas which column to use, the library generates a numeric index starting from 0.

In [3]:
```
url = 'https://raw.githubusercontent.com/GiorgioBar/pandas/main/datasets/area.csv'
area_df = pd.read_csv(url)
area_df.head()
```

Out[3]:

	ITTER107	Territory	TIPO_DATO4	Data type	Value
0	IT	Italy	TOTAREA	total area (Ha)	3.020683e+07
1	IT	Italy	TOTAREA2	total area (km2)	3.020683e+05
2	ITC	Nord-ovest	TOTAREA	total area (Ha)	5.792680e+06
3	ITC	Nord-ovest	TOTAREA2	total area (km2)	5.792680e+04
4	ITC1	Piemonte	TOTAREA	total area (Ha)	2.538670e+06

Taking a first look at the dataframe

We can get the first *N* rows of a DataFrame using the *head()* function. Similarly, we can get the last *N* rows using the *tail()* function.

In [4]:
```
population_df.head(3)
```

Out[4]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
0	IT	Italy	1	males	Y0	0 years	205956
1	IT	Italy	2	females	Y0	0 years	195565
2	IT	Italy	9	total	Y0	0 years	401521

In [5]:
```
population_df.tail(3)
```

Out[5]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
41307	IT111	Sud Sardegna	2	females	Y23	23 years	1251
41308	IT111	Sud Sardegna	1	males	TOTAL	total	165713
41309	IT111	Sud Sardegna	9	total	Y20	20 years	2784

We can inquire about the number of rows and columns in the DataFrame and extract a row by its index position (which starts counting at 0). This method returns an object of class **Series**, a one-dimensional labeled array of values.

In [6]:
```
print(population_df.shape)
print(population_df.iloc[2])
```

```
(41310, 7)
ITTER107          IT
Territory      Italy
SEXISTAT1          9
Gender         total
ETA1              Y0
Age          0 years
Value         401521
Name: 2, dtype: object
```

Perform SQL-like operations using pandas

SELECT

We can select some columns of a DataFrame by passing a list of column names to the indexing operator ([ ]) of the DataFrame.

In [7]:
```
# print(population_df.columns)
population_df[['Territory', 'Gender', 'Age', 'Value']].head(3)
```

Out[7]:

	Territory	Gender	Age	Value
0	Italy	males	0 years	205956
1	Italy	females	0 years	195565
2	Italy	total	0 years	401521

We can add a calculated column using the assign() method of the DataFrame, this method returns a new DataFrame with the new columns in addition to the existing ones.

In [8]:
```
population_df.assign(Value2=population_df['Value'] / 1000)
```

Out[8]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value	Value2
0	IT	Italy	1	males	Y0	0 years	205956	205.956
1	IT	Italy	2	females	Y0	0 years	195565	195.565
2	IT	Italy	9	total	Y0	0 years	401521	401.521
3	ITC	Nord-ovest	1	males	Y0	0 years	54122	54.122
4	ITC	Nord-ovest	2	females	Y0	0 years	51309	51.309
...	...	...	...	...	...	...	...	...
41305	IT111	Sud Sardegna	1	males	Y25	25 years	1556	1.556
41306	IT111	Sud Sardegna	2	females	Y91	91 years	728	0.728
41307	IT111	Sud Sardegna	2	females	Y23	23 years	1251	1.251
41308	IT111	Sud Sardegna	1	males	TOTAL	total	165713	165.713
41309	IT111	Sud Sardegna	9	total	Y20	20 years	2784	2.784

41310 rows × 8 columns

In [9]:
```
population_df.assign(ETA2=population_df['ETA1']
                     .str.replace("Y", "", regex = False)
                     .replace("_GE100", "100", regex = False)
                     .replace("TOTAL", "999", regex = False)
                     .astype(int))
```

Out[9]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value	ETA2
0	IT	Italy	1	males	Y0	0 years	205956	0
1	IT	Italy	2	females	Y0	0 years	195565	0
2	IT	Italy	9	total	Y0	0 years	401521	0
3	ITC	Nord-ovest	1	males	Y0	0 years	54122	0
4	ITC	Nord-ovest	2	females	Y0	0 years	51309	0
...	...	...	...	...	...	...	...	...
41305	IT111	Sud Sardegna	1	males	Y25	25 years	1556	25
41306	IT111	Sud Sardegna	2	females	Y91	91 years	728	91
41307	IT111	Sud Sardegna	2	females	Y23	23 years	1251	23
41308	IT111	Sud Sardegna	1	males	TOTAL	total	165713	999
41309	IT111	Sud Sardegna	9	total	Y20	20 years	2784	20

41310 rows × 8 columns

ORDER BY

We can sort a DataFrame by multiple columns in different orders using the sort_values method. Notice that by default this method returns a new DataFrame object, leaving the original data intact. Most pandas operations return a copy of the Series or DataFrame object, we can either assign this object to a new variable or overwrite the original one. The usage of the *inplace=True* keyword argument, available for some methods, is discouraged.

In [10]:
```
population_df.sort_values(by = "Value", ascending = False).head(3)
# population_df.sort_values(by = ["Age", "Value"], ascending= [True, False]).head(
```

Out[10]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
40604	IT	Italy	9	total	TOTAL	total	58983122
40603	IT	Italy	2	females	TOTAL	total	30235705
40602	IT	Italy	1	males	TOTAL	total	28747417

WHERE

DataFrames can be filtered using boolean indexing, passing a Series of boolean values to the indexing operator.

In [11]:
```
# population_df[population_df["Value"] > 50000000]

is_less_than_one_year_old = population_df["Age"] == "0 years"
print(is_less_than_one_year_old.value_counts())
```

```
False    40905
True       405
Name: Age, dtype: int64
```

In [12]:
```
population_df[is_less_than_one_year_old]
```

Out[12]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
0	IT	Italy	1	males	Y0	0 years	205956
1	IT	Italy	2	females	Y0	0 years	195565
2	IT	Italy	9	total	Y0	0 years	401521
3	ITC	Nord-ovest	1	males	Y0	0 years	54122
4	ITC	Nord-ovest	2	females	Y0	0 years	51309
...	...	...	...	...	...	...	...
400	IT110	Barletta-Andria-Trani	2	females	Y0	0 years	1356
401	IT110	Barletta-Andria-Trani	9	total	Y0	0 years	2844
41183	IT111	Sud Sardegna	1	males	Y0	0 years	864
41216	IT111	Sud Sardegna	2	females	Y0	0 years	785
41230	IT111	Sud Sardegna	9	total	Y0	0 years	1649

405 rows × 7 columns

We can specify multiple conditions using | (OR) and & (AND) operators and enclosing each condition inside a pair of parentheses.

In [13]:
```
population_df[(population_df["Age"] == "0 years") & (population_df["Value"] > 1000
```

Out[13]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
0	IT	Italy	1	males	Y0	0 years	205956
1	IT	Italy	2	females	Y0	0 years	195565
2	IT	Italy	9	total	Y0	0 years	401521
5	ITC	Nord-ovest	9	total	Y0	0 years	105431

In [14]:
```
italy_population_df = population_df[ (population_df["Gender"] == "total") & (popul
italy_population_df
```

Out[14]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
2	IT	Italy	9	total	Y0	0 years	401521
5	ITC	Nord-ovest	9	total	Y0	0 years	105431
8	ITC1	Piemonte	9	total	Y0	0 years	26804
11	ITC11	Torino	9	total	Y0	0 years	13938
14	ITC12	Vercelli	9	total	Y0	0 years	1002
...	...	...	...	...	...	...	...
41292	IT111	Sud Sardegna	9	total	Y61	61 years	5343
41295	IT111	Sud Sardegna	9	total	Y19	19 years	2869
41297	IT111	Sud Sardegna	9	total	Y98	98 years	128
41299	IT111	Sud Sardegna	9	total	Y86	86 years	1819
41309	IT111	Sud Sardegna	9	total	Y20	20 years	2784

13635 rows × 7 columns

In [15]:

```
macroregions_population_df = population_df[ (population_df["Gender"] == "total") &
                                            (population_df["Age"] != "total") &
                                            ((population_df["ITTER107"] == "ITC") |
                                             (population_df["ITTER107"] == "ITD") |
                                             (population_df["ITTER107"] == "ITE") |
                                             (population_df["ITTER107"] == "ITF") |
                                             (population_df["ITTER107"] == "ITG") )
macroregions_population_df
```

Out[15]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
5	ITC	Nord-ovest	9	total	Y0	0 years	105431
92	ITD	Nord-est	9	total	Y0	0 years	79817
179	ITE	Centro (I)	9	total	Y0	0 years	74555
257	ITF	Sud	9	total	Y0	0 years	96229
347	ITG	Isole	9	total	Y0	0 years	45489
...	...	...	...	...	...	...	...
40213	ITD	Nord-est	9	total	Y_GE100	100 years and over	4463
40279	ITC	Nord-ovest	9	total	Y_GE100	100 years and over	5421
40379	ITE	Centro (I)	9	total	Y_GE100	100 years and over	4503
40457	ITF	Sud	9	total	Y_GE100	100 years and over	3894
40547	ITG	Isole	9	total	Y_GE100	100 years and over	1878

505 rows × 7 columns

GROUP BY

We can group DataFrame rows into buckets, based on shared values in a given column or columns, by invoking the *groupby* method on a DataFrame. The method returns a

DataFrameGroupBy object, a storage container that provides a set of methods to analyze each independent group.

In [16]:
```
# territory_groups = italy_population_df.groupby("ITTER107")
territory_groups = italy_population_df.groupby(["ITTER107", "Territory"])
age_groups = macroregions_population_df.groupby(["Age"])
```

To know in advance in how many different groups the data will be divided into, we can use the *nunique()* function on any column, which gives us the number of unique values in that column. We can also apply the *unique()* function on a column, which returns unique values of a Series object.

In [17]:
```
# Series.unique() Returns unique values of a Series object (as a NumPy array)

# print(population_df["Age"].unique())
# print(population_df["Age"].unique().size)

# print(population_df["ITTER107"].unique())
print(population_df["ITTER107"].unique().size)
print(population_df["Territory"].unique().size)
```

```
135
134
```

The DataFrameGroupBy object stores a group for each unique value in the column we specify. Its *ngroups* method returns the number of groups, while its *size* method returns a Series with a list of the groups and their row counts. The *get_group* method accepts a group name and returns a DataFrame with the corresponding rows.

In [18]:
```
print(territory_groups.ngroups)
print(territory_groups.size())
# territory_groups.get_group("IT")
# Since we are grouping by two columns, the get_group method requires a tuple of va
territory_groups.get_group(("IT", "Italy"))
```

```
135
ITTER107  Territory
IT        Italy                    101
IT108     Monza e della Brianza    101
IT109     Fermo                    101
IT110     Barletta-Andria-Trani    101
IT111     Sud Sardegna             101
                                  ...
ITG2      Sardegna                 101
ITG25     Sassari                  101
ITG26     Nuoro                    101
ITG27     Cagliari                 101
ITG28     Oristano                 101
Length: 135, dtype: int64
```

Out[18]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
2	IT	Italy	9	total	Y0	0 years	401521
551	IT	Italy	9	total	Y1	1 years	405952
1087	IT	Italy	9	total	Y2	2 years	424329
1222	IT	Italy	9	total	Y3	3 years	444659
1728	IT	Italy	9	total	Y4	4 years	465225
...	...	...	...	...	...	...	...
38714	IT	Italy	9	total	Y96	96 years	41709
38996	IT	Italy	9	total	Y97	97 years	29970
39398	IT	Italy	9	total	Y98	98 years	21142
39864	IT	Italy	9	total	Y99	99 years	14144
40276	IT	Italy	9	total	Y_GE100	100 years and over	20159

101 rows × 7 columns

In [19]:

```
print(italy_population_df["Age"].nunique())
print(age_groups.ngroups)
print(age_groups.size())
age_groups.get_group("0 years")
```

```
101
101
Age
0 years               5
1 years               5
10 years              5
100 years and over    5
11 years              5
                     ..
95 years              5
96 years              5
97 years              5
98 years              5
99 years              5
Length: 101, dtype: int64
```

Out[19]:

	ITTER107	Territory	SEXISTAT1	Gender	ETA1	Age	Value
5	ITC	Nord-ovest	9	total	Y0	0 years	105431
92	ITD	Nord-est	9	total	Y0	0 years	79817
179	ITE	Centro (I)	9	total	Y0	0 years	74555
257	ITF	Sud	9	total	Y0	0 years	96229
347	ITG	Isole	9	total	Y0	0 years	45489

We can invoke methods on the GroupBy object to apply aggregate operations to every group. By default, pandas includes all numerical columns, we can target a single column by passing its name inside square brackets after the GroupBy object.

In [20]:
```
# territory_groups.sum()
territory_groups["Value"].sum()
```

Out[20]:
```
ITTER107  Territory
IT        Italy                    58983122
IT108     Monza e della Brianza      870112
IT109     Fermo                      168485
IT110     Barletta-Andria-Trani      379251
IT111     Sud Sardegna               335108
                                     ...
ITG2      Sardegna                  1579181
ITG25     Sassari                    474142
ITG26     Nuoro                      199349
ITG27     Cagliari                   419770
ITG28     Oristano                   150812
Name: Value, Length: 135, dtype: int64
```

In [21]:
```
age_groups["Value"].sum()
```

Out[21]:
```
Age
0 years               401521
1 years               405952
10 years              542130
100 years and over     20159
11 years              557902
                       ...
95 years               57139
96 years               41709
97 years               29970
98 years               21142
99 years               14144
Name: Value, Length: 101, dtype: int64
```

JOIN

We can perform a JOIN using the *merge* method.

In [24]:
```
print(macroregions_population_df.shape)
print(area_df.shape)

area_in_km2_df = area_df[area_df["TIPO_DATO4"] == "TOTAREA2"]
print(area_in_km2_df.shape)

merge_df = macroregions_population_df.merge(area_in_km2_df, how = "inner", on = "I
merge_df
```

```
(505, 7)
(270, 5)
(135, 5)
```

Out[24]:

	ITTER107	Territory_x	SEXISTAT1	Gender	ETA1	Age	Value_x	Territory_y	TIPO_DATO4
0	ITC	Nord-ovest	9	total	Y0	0 years	105431	Nord-ovest	TOTAREA2
1	ITC	Nord-ovest	9	total	Y1	1 years	106563	Nord-ovest	TOTAREA2
2	ITC	Nord-ovest	9	total	Y2	2 years	112013	Nord-ovest	TOTAREA2
3	ITC	Nord-ovest	9	total	Y3	3 years	117051	Nord-ovest	TOTAREA2
4	ITC	Nord-ovest	9	total	Y4	4 years	123073	Nord-ovest	TOTAREA2
...	...	...	...	...	...	...	...	...	...
500	ITG	Isole	9	total	Y96	96 years	3853	Isole	TOTAREA2
501	ITG	Isole	9	total	Y97	97 years	2908	Isole	TOTAREA2
502	ITG	Isole	9	total	Y98	98 years	1851	Isole	TOTAREA2
503	ITG	Isole	9	total	Y99	99 years	1280	Isole	TOTAREA2
504	ITG	Isole	9	total	Y_GE100	100 years and over	1878	Isole	TOTAREA2

505 rows × 11 columns

In [25]:

```
population_density_df = merge_df[['ITTER107', 'Territory_x', 'Age', 'Value_x', 'Va
population_density_df = population_density_df.rename(columns = {'Territory_x':'Ter
population_density_df = population_density_df.assign(Density=population_density_df
population_density_df
```

Out[25]:

	ITTER107	Territory	Age	Population	Area	Density
0	ITC	Nord-ovest	0 years	105431	57926.8010	1.820073
1	ITC	Nord-ovest	1 years	106563	57926.8010	1.839615
2	ITC	Nord-ovest	2 years	112013	57926.8010	1.933699
3	ITC	Nord-ovest	3 years	117051	57926.8010	2.020671
4	ITC	Nord-ovest	4 years	123073	57926.8010	2.124630
...	...	...	...	...	...	...
500	ITG	Isole	96 years	3853	49931.9984	0.077165
501	ITG	Isole	97 years	2908	49931.9984	0.058239
502	ITG	Isole	98 years	1851	49931.9984	0.037070
503	ITG	Isole	99 years	1280	49931.9984	0.025635
504	ITG	Isole	100 years and over	1878	49931.9984	0.037611

505 rows × 6 columns

In [ ]:

```
%%shell
jupyter nbconvert --to html /content/SQL_like_queries_using_Pandas.ipynb
```

```
[NbConvertApp] Converting notebook /content/SQL_like_queries_using_Pandas.ipynb to
html
[NbConvertApp] Writing 710260 bytes to /content/SQL_like_queries_using_Pandas.html
```

Out[ ]:

In [ ]:

In [ ]:

```
from google.colab import drive
drive.mount('/content/drive')
```